

TABsucks

软件配置与运维文档

Configuration Management and Operations Plan

文档编号	TABSUCKS-CMO-001
文档版本	V1.0
文档状态	课程提交候选版
项目团队	TABsucks 项目组
组长学号/姓名	待填写
编制日期	2026-07-18

适用范围：TABsucks Windows 桌面发行与本地 Web 运行环境

目录

文档控制	4
修订记录	4
审批与职责	4
1. 引言	4
1.1 编写目的	4
1.2 适用范围	4
1.3 项目运行形态	5
2. 管理原则与配置策略	5
2.1 管理目标	5
2.2 现状、基线与规划的标识	5
3. 软件配置管理	5
3.1 配置项识别	5
3.2 配置项命名	6
3.3 配置基线	6
3.4 变更控制流程	6
4. 版本控制与协作管理	7
4.1 仓库与分支	7
4.2 提交和 Pull Request 规范	7
4.3 忽略规则和大文件治理	8
5. 环境与依赖配置	8
5.1 环境分层	8
5.2 依赖清单职责	8
5.3 FFmpeg 管理	9
5.4 敏感配置	9
6. 持续集成与质量门禁	9
6.1 当前已实现能力	9
6.2 当前差距	9
6.3 目标 CI 流水线	9
6.4 发布质量门禁	10
7. 构建与制品管理	10
7.1 构建链	10
7.2 拆分发行策略	10
7.3 安装顺序	11
7.4 制品仓库规则	11

8. 部署与安装	11
8.1 部署拓扑	11
8.2 安装位置与权限	11
8.3 启动流程	11
8.4 升级和回滚	12
9. 运行数据、备份与恢复	12
9.1 数据目录	12
9.2 持久化控制	12
9.3 备份策略	13
9.4 数据保留和清理	13
10. 可观测性与运行维护	13
10.1 日志	13
10.2 健康检查	13
10.3 服务目标	14
10.4 容量与资源管理	14
11. 事件、问题和缺陷管理	14
11.1 事件分级	14
11.2 标准处置流程	14
11.3 常见问题处理	15
12. 安全、合规与供应链	15
12.1 本地服务安全	15
12.2 软件供应链	15
12.3 隐私	16
13. 发布、回滚与运维检查表	16
13.1 发布前	16
13.2 发布中	16
13.3 发布后	16
14. 角色分工与 RACI	17
15. 风险登记与持续改进	17
16. 当前结论	18
附录 A: 关键路径	18
附录 B: 版本发布记录模板	18
附录 C: 配置审计证据	18

文档控制

修订记录

版本	日期	修订人	修订内容	状态
V0.1	2026-07-17	项目组	建立 Windows 桌面发行、依赖和日志管理基础	草稿
V0.9	2026-07-18	项目组	增加 CPU/GPU/模型拆分发行业、运行时验证和发布约束	评审稿
V1.0	2026-07-18	项目组	按大型软件团队标准形成配置与运维基线	提交候选

审批与职责

角色	主要职责	审批对象
项目负责人	批准版本范围、发布窗口和重大风险处置	正式发布基线
配置管理员	维护配置项、分支、标签、制品和变更记录	配置基线
开发负责人	评审技术变更及依赖兼容性	源码与构建配置
测试负责人	确认测试准入和遗留缺陷风险	发布测试报告
发布负责人	构建、校验、上传、回滚演练	Release 制品
运维支持负责人	日志分析、故障分级、用户支持	运维记录

注：课程项目中一名成员可以兼任多个角色，但审批责任和操作责任应在发布记录中明确，避免“开发者自行修改、自行批准、自行发布”而无法追溯。

1. 引言

1.1 编写目的

本文档规定 TABsucks 项目的软件配置管理、版本控制、持续集成、构建发布、部署安装、运行维护、故障处置、备份恢复及持续改进要求。目标是使项目在多人协作、频繁模型迭代和 Windows 大体积发行场景下仍具备可重复构建、可验证发布、可追踪变更和可恢复运行的能力。

本文档既描述截至 2026-07-18 已在仓库和本地发行流程中实现的能力，也列出尚未完成但应纳入后续演化的控制措施。所有规划项均明确标记，不作为已经完成的项目成果。

1.2 适用范围

本规范适用于：

- `src/` 下的 Python 后端、内核、插件和 Web UI 代码；
- `tests/` 下的 Python 与 JavaScript 测试；
- `requirements*.txt`、`constraints-windows.txt` 和 `pyproject.toml`；
- `scripts/` 下的开发、构建和运行时验证脚本；
- `packaging/` 下的 PyInstaller、Inno Setup 和 FFmpeg 发行资源；
- BS-RoFormer、ChordMini、ISMIR2019 等模型和算法资源；

- Windows CPU 基础包、模型包和 GPU/CUDA 升级包；
- GitHub 仓库、Pull Request、Actions、Tag 和 Release；
- `%LOCALAPPDATA%\TABsucks` 下的用户车间、缓存和日志。

1.3 项目运行形态

TABsucks 是本地部署的智能音乐分析应用。桌面启动器启动 FastAPI/Uvicorn 服务，默认仅监听 `127.0.0.1`，服务就绪后自动打开浏览器。核心功能包括音频导入、音轨分离、和弦与节奏分析、车间持久化、可视化和 MIDI 导出等。

系统采用“本地 Web 界面 + Python 微内核 + 插件”的形态。最终用户通过 Windows EXE 使用软件，无需单独安装 Python。AI 模型、CPU 运行环境和 GPU 运行环境体积差异较大，因此发行物采用分层交付。

2. 管理原则与配置策略

2.1 管理目标

配置与运维工作遵循以下目标：

- 1 **唯一性：** 每个正式版本必须能由 Tag、提交哈希和制品校验值唯一定位。
- 2 **可重复性：** 相同源码、依赖约束和构建工具应产生功能等价的发行物。
- 3 **最小权限：** 软件安装在当前用户目录，服务只监听本机，不要求管理员权限。
- 4 **职责分离：** 代码实现、质量确认和正式发布至少需要两个角色参与确认。
- 5 **制品不可变：** 已发布资产不得同名覆盖；修复后必须提升版本并重新发布。
- 6 **数据隔离：** 程序文件、模型文件和用户数据分别管理，升级不应破坏用户车间。
- 7 **可观测性：** 启动、运行和退出过程应写入 UTF-8 文件日志并在状态窗口显示。
- 8 **故障可恢复：** 状态采用原子写入、周期保存和安全退出，严重故障支持回退上一版本。

2.2 现状、基线与规划的标识

标识	含义	使用规则
已实现	已存在于源码、脚本或仓库配置，并有直接证据	可作为项目成果
已验证	已在指定环境执行测试或运行时检查	必须记录环境和结果
受限	已实现但存在环境、体积、兼容性或流程限制	发布说明必须披露
规划	尚未落地的改进措施	不计入当前完成能力

3. 软件配置管理

3.1 配置项识别

项目配置项采用“源码、环境、资源、文档、制品、运行数据”六类管理。

配置项类别	主要内容	权威位置	变更控制
应用源码	内核、API、插件、音频、前端静态资源	<code>src/</code>	Git + PR

配置项类别	主要内容	权威位置	变更控制
测试资产	单元测试、HTTP 测试、JavaScript 测试	tests/	Git + 测试评审
依赖与工具配置	requirements、constraints、pyproject	仓库根目录	依赖评审
构建与安装配置	PyInstaller、Inno Setup、PowerShell 脚本	packaging/、scripts/	发布负责人评审
模型与媒体运行资源	模型权重、YAML、FFmpeg/FFprobe	模型包、packaging/ffmpeg/	来源、版本、校验值
项目文档	架构、开发、发行、开发日志	docs/	文档评审
构建制品	EXE、BIN、SHA256SUMS	GitHub Release	不可变发布
用户运行数据	车间状态、音频缓存、分析结果、日志	%LOCALAPPDATA%\TABsucks	本地生命周期管理

3.2 配置项命名

正式发行资产使用统一命名：

TABsucks-Base-Setup.exe
TABsucks-GPU-Upgrade-Setup.exe
TABsucks-GPU-Upgrade-Setup-1.bin
TABsucks-GPU-Upgrade-Setup-2.bin
TABsucks-Models-Setup.exe
SHA256SUMS.txt

后续正式发布建议在文件名中增加语义化版本和平台：

TABsucks-0.2.0-windows-x64-base.exe
TABsucks-0.2.0-windows-x64-models.exe
TABsucks-0.2.0-windows-x64-gpu-upgrade.exe

3.3 配置基线

项目设置四类基线：

基线	形成条件	主要内容	变更要求
开发基线	功能分支通过局部测试	源码、测试、开发文档	PR 审查
集成基线	合并到集成/主分支	可协同运行的完整源码	回归测试
发布候选基线	构建和测试准入通过	Tag 候选、依赖清单、安装包	发布评审
正式发布基线	Release 上传并复核	Tag、制品、校验值、发行说明	禁止覆盖

截至本文档编制时，Windows 拆分发行业处于“发布候选准备”阶段：CPU 基础便携目录及 CPU/GPU 运行时已验证，基础包和模型包已生成；最终 GPU 分片因打包过程暂停，需要重新生成并完成安装链验证后，才能转为正式发布基线。

3.4 变更控制流程

所有影响用户行为、数据格式、依赖、模型或发行物的变更应执行：

- 1 建立问题、任务或变更说明，描述动机、范围、风险和验收条件。
- 2 从受控基线创建功能分支。
- 3 实现代码、测试和必要文档。
- 4 执行静态检查、单元测试和受影响流程测试。
- 5 创建 Pull Request，由非作者成员审查。
- 6 对依赖、状态文件或接口变化执行兼容性评审。
- 7 合并后建立新的集成基线。
- 8 发布负责人依据检查表构建，不允许直接使用来源不明的本地文件。
- 9 发布后记录 Tag、提交哈希、构建环境和 SHA-256。

紧急修复同样需要保留问题记录和审查记录。若确需先修复后补文档，应在一个工作日内补齐变更说明和回归结果。

4. 版本控制与协作管理

4.1 仓库与分支

项目使用 Git 进行版本控制，GitHub 作为远程协作和发行平台。仓库已有主分支、功能分支和阶段性结束分支，提交历史能够反映音频输入、资源控制、和弦识别、UI、车间缓存和分析链路等并行开发过程。

建议长期采用以下分支模型：

分支类型	示例	生命周期
主分支	<code>main</code>	始终保持可集成、可发布
功能分支	<code>feature/p1-workshop-cache-system</code>	功能完成并合并后删除
修复分支	<code>fix/bass-root-stereo-input</code>	缺陷关闭后删除
发布分支	<code>release/0.2.0</code>	发布稳定期短期存在
紧急修复	<code>hotfix/0.2.1-startup</code>	从正式 Tag 创建

4.2 提交和 Pull Request 规范

提交应保持单一目的，推荐使用 `feat`、`fix`、`test`、`docs`、`refactor`、`build`、`ci`、`chore` 等类型。

Pull Request 至少包含：

- 变更目的与用户影响；
- 主要实现说明；
- 测试范围与结果；
- 兼容性和数据迁移影响；
- 截图或日志证据（适用于 UI、运行和打包变更）；
- 已知限制及回滚方式。

主分支保护建议设置为：禁止直接推送、至少一名审查者批准、必需检查通过、分支必须与目标分支同步、禁止强制推送。

4.3 忽略规则和大文件治理

`.gitignore` 已排除虚拟环境、缓存、日志、音频、模型权重、`dist/`、`build/` 和 `.build/`。这是为了防止：

- 将开发者本机环境误当作项目依赖；
- 大型模型和构建产物拖慢克隆与审查；
- 日志和用户音频造成隐私泄露；
- 不同机器生成的二进制污染源码历史。

模型权重不能只依靠文件扩展名识别。正式模型包应维护清单，至少记录模型名称、功能、来源、许可证、文件大小、SHA-256、适用程序版本和更新日期。

5. 环境与依赖配置

5.1 环境分层

环境	用途	关键要求
开发环境	源码运行、调试、测试	Windows x64、Python 3.10、虚拟环境
CPU 构建环境	构建基础包	CPU-only PyTorch、ONNX Runtime CPU
GPU 构建环境	构建 GPU 升级包	PyTorch CUDA 12.4、ONNX Runtime GPU
用户环境	安装并使用软件	Windows x64；GPU 用户需兼容 NVIDIA 驱动

已验证的 Windows GPU 构建环境为 Python 3.10.18、PyTorch 2.6.0+cu124、ONNX Runtime GPU 1.23.2。隔离 CPU 构建环境使用 PyTorch 2.6.0+cpu 和 ONNX Runtime 1.23.2。

5.2 依赖清单职责

文件	职责
<code>requirements-base.txt</code>	CPU/GPU 共用的直接运行依赖
<code>requirements.txt</code>	默认 CPU 源码运行环境
<code>requirements-gpu.txt</code>	NVIDIA GPU 源码运行环境
<code>requirements-dev.txt</code>	测试、质量检查和 Windows 构建工具
<code>requirements-audio-device.txt</code>	可选的 Python 原生音频设备播放
<code>constraints-windows.txt</code>	当前 GPU Windows 环境已验证版本
<code>pyproject.toml</code>	Ruff、Black、isort、mypy 和 pytest 配置

依赖升级必须说明升级原因，重点验证 NumPy、Librosa、PyTorch、ONNX Runtime、audio-separator、FastAPI 和 PyInstaller 的兼容性。禁止同时安装互斥的 CPU/GPU ONNX Runtime 后端，防止 Provider 冲突。

5.3 FFmpeg 管理

源码调试时可使用系统 `PATH` 中的 FFmpeg。Windows 发行版必须携带 `ffmpeg.exe` 和 `ffprobe.exe`，避免用户首次启动时访问 GitHub 下载而失败。构建脚本先检查本机路径，必要时由准备脚本获取资源；正式构建前应固定 FFmpeg 版本并记录校验值。

5.4 敏感配置

密钥和令牌不得提交仓库。GitHub Actions 使用仓库 Secrets，例如 AI 评审所需的 API Key。开发环境中的密钥应通过环境变量或本地 `.env` 管理，`.env` 已被 Git 忽略。日志不得输出完整令牌、用户隐私路径或第三方认证信息。

6. 持续集成与质量门禁

6.1 当前已实现能力

仓库当前存在两类 GitHub Actions：

- **CodeQL**：在 `main` 的 push、Pull Request 及每周计划任务上执行 Python 静态安全分析。
- **AI Code Review**：在 Pull Request 打开、同步或重新打开时执行自动化评审并提交评论。

本地工程化能力包括 Ruff、Black、isort、mypy、pytest、PyInstaller 构建脚本、Inno Setup 安装包脚本及运行时 Provider 验证脚本。

6.2 当前差距

仓库尚未建立完整的普通 CI 流水线，现有 Actions 不等同于“自动测试通过”。当前缺少：

- 每次 PR 自动执行 Ruff/格式检查；
- Python 单元测试和覆盖率；
- JavaScript 测试；
- 依赖安装可重复性验证；
- Windows CPU 构建烟雾测试；
- Release Tag 自动生成校验清单；
- 构建制品签名和来源证明。

因此，发布前仍必须执行人工控制的本地测试和构建检查。

6.3 目标 CI 流水线

规划的流水线分为快速反馈、集成验证和正式发布三个层次：

- 1 **PR 快速检查**：Ruff、Black、isort、mypy、关键单元测试，目标 10 分钟内完成。
- 2 **主分支集成**：完整 Python 测试、JavaScript 测试、覆盖率、CodeQL 和依赖审计。
- 3 **发布流水线**：Windows CPU 便携版构建、启动烟雾测试、安装包编译、SHA-256 和 Release 草稿。
- 4 **GPU 发布**：由于体积和硬件要求，初期采用受控自托管 GPU Runner 或人工构建，并保留运行时验证 JSON。

6.4 发布质量门禁

发布候选必须同时满足：

- 工作区对应唯一提交，无未记录的源码改动；
- 所有必需配置和模型资源存在；
- 静态检查无阻断问题；
- 受影响测试和核心回归测试通过；
- CPU 包中不存在 CUDA 文件和独立模型；
- GPU 运行时报告显示 CUDA 版本符合基线；
- ONNX Runtime Provider 与包类型一致；
- 安装、启动、浏览器打开、退出和日志路径通过烟雾测试；
- 所有 Release 资产小于 GitHub 单文件 2 GiB 限制；
- SHA-256 清单生成并由第二人复核；
- 已知缺陷、硬件要求和安装顺序写入发行说明。

7. 构建与制品管理

7.1 构建链

Windows 构建链如下：

受控源码与依赖

- > 准备 FFmpeg 和模型资源
- > PyInstaller onedir 构建
- > CPU/GPU 运行时检查
- > Inno Setup 编译与分片
- > 安装链烟雾测试
- > SHA-256 校验清单
- > Git Tag 和 GitHub Release

PyInstaller 使用 onedir 布局，TABSucks.exe 与 _internal 目录共同构成程序，禁止只复制 EXE。
packaging/tabsucks.spec 支持通过环境变量控制是否包含模型以及发行目录名称。

7.2 拆分发行策略

包	内容	目标用户	当前典型体积
CPU 基础包	程序、CPU 推理环境、FFmpeg、基础功能	全部用户	安装包约 242 MiB
模型资源包	BS-RoFormer、ChordMini 等权重	需要模型功能的用户	约 402 MiB
GPU 升级包	CUDA PyTorch、GPU Provider、运行依赖	NVIDIA GPU 用户	完整数据约 3 GiB，分片发布

GPU 包采用 Inno Setup 磁盘分片，每个 .bin 小于 2 GiB。启动 EXE 与全部 BIN 必须位于同一目录。
GPU 包覆盖基础包的完整 _internal 运行环境，而不是只替换 Torch/CUDA 子目录；后者在实际验证中会造成启动异常。

7.3 安装顺序

1. TABsucks-Base-Setup.exe
2. TABsucks-GPU-Upgrade-Setup.exe (可选, 仅 NVIDIA GPU)
3. TABsucks-Models-Setup.exe (需要模型功能时)

GPU 包会删除并替换 `_internal`, 所以必须先于模型包安装。安装程序已加入基础程序存在性检查和错误顺序检查。

7.4 制品仓库规则

GitHub Release 是正式二进制制品仓库, Git 仓库仅保存可审查的源代码与构建定义。Release 应包含:

- 完整安装文件;
- `SHA256SUMS.txt`;
- 版本说明和已知问题;
- 最低系统要求;
- 安装和升级顺序;
- 对应 Tag 和提交哈希;
- 模型及第三方组件许可证说明。

任何上传失败或中断产生的文件均视为临时制品, 必须删除后重新生成, 不得通过补传缺失分片将其声明为正式版本。

8. 部署与安装

8.1 部署拓扑

TABsucks 采用单机、单用户、本地服务部署:

Windows 用户
-> TABsucks 启动状态窗口
-> FastAPI/Uvicorn 本地服务 (127.0.0.1:8000 或后续可用端口)
-> 默认浏览器 Web UI
-> Kernel / PluginManager / AnalysisEngine / ResourceController
-> 本地模型、缓存、音频和状态文件

8.2 安装位置与权限

基础程序默认安装至:

`%LOCALAPPDATA%\Programs\TABsucks`

安装使用当前用户权限, 不需要修改系统级 Python 环境。开始菜单快捷方式默认创建, 桌面快捷方式可选。程序和用户数据分离, 降低升级、卸载或权限错误造成的数据损失。

8.3 启动流程

- 1 创建用户缓存和日志目录。
- 2 配置 UTF-8 文件日志, 并将标准输出和标准错误纳入日志。
- 3 优先使用发行版携带的 FFmpeg。

- 4 设置模型缓存和内置模型路径。
- 5 启动 Kernel，加载可恢复车间及插件。
- 6 尝试监听 `127.0.0.1:8000`；若端口占用，向后寻找可用端口。
- 7 后台启动 Uvicorn 并轮询 HTTP 就绪状态。
- 8 服务就绪后显示地址并自动打开浏览器。
- 9 用户关闭窗口时停止服务、保存车间并释放 Kernel。

8.4 升级和回滚

升级前应记录现有版本并备份重要车间。基础包、GPU 包和模型包必须使用同一兼容版本矩阵。若新版本发生严重故障：

- 1 保存 `%LOCALAPPDATA%\TABsucks\cache` 和 `logs`。
- 2 卸载或覆盖安装上一稳定版本基础包。
- 3 按对应版本重新安装 GPU 和模型包。
- 4 启动后验证车间加载、模型 Provider 和核心分析流程。
- 5 将失败日志、版本号和复现步骤关联到缺陷记录。

状态文件使用相对路径以提高跨目录恢复能力，但不同版本的数据结构兼容性仍需通过迁移测试保证。未来若修改 `state.json` 模式，应增加显式 schema 版本和迁移器。

9. 运行数据、备份与恢复

9.1 数据目录

```
%LOCALAPPDATA%\TABsucks\cache
%LOCALAPPDATA%\TABsucks\logs
```

缓存目录保存车间、原始音频、分离轨道、分析结果和模型缓存。日志目录保存按启动时间命名的 UTF-8 日志。

9.2 持久化控制

车间状态使用 `state.json` 管理，关键控制包括：

- 仅保存相对车间目录的资源路径；
- 先写临时文件，再使用 `os.replace` 原子替换；
- 默认每 5 秒检查脏状态并自动保存；
- 切换或关闭车间时执行保存；
- Kernel 退出时停止自动保存线程并保存全部车间；
- 删除车间时可将 `state.json` 备份到 `recycle_bin`；
- 损坏或缺失的状态文件被跳过，不阻塞其他车间加载。

当前机制降低了异常退出损失，但缓存目录仍不是完整备份系统。

9.3 备份策略

课程版本建议执行：

- 重要演示前手工复制整个 `cache` 目录；
- 保留最近一次稳定版本安装包和模型包；
- 关键发布时保存 `SHA256SUMS.txt` 和运行时验证 JSON；
- 用户上传的原始音频如不可重新获取，应另行备份。

面向生产化的规划策略为“每日增量、每周完整、保留四周”，并增加备份恢复演练。备份必须与软件版本关联，恢复后验证 `state.json`、音频文件和分析结果引用的一致性。

9.4 数据保留和清理

当前卸载不主动删除用户缓存与日志，以避免误删成果。后续应提供显式的“清理缓存”“导出车间”“删除全部用户数据”操作，并设置日志轮转：

- 单日志建议上限 20 MiB；
- 保留最近 14 天或最近 20 个日志文件；
- 崩溃日志和用户标记日志可延长保留；
- 清理前不得删除正在使用的车间和模型。

以上轮转策略属于规划项，当前版本尚未自动执行。

10. 可观测性与运行维护

10.1 日志

启动窗口实时显示运行日志，并提供“打开日志目录”按钮。文件日志格式包含时间、级别、Logger 和消息：

```
2026-07-18 10:30:00,000 [INFO] tabsucks.desktop: TABsucks 已就绪
```

运维分析优先关注：

- 启动路径、日志路径和服务地址；
- 插件注册与加载失败；
- 车间加载失败；
- FFmpeg 和下载错误；
- CPU/GPU 请求设备与实际执行设备；
- 模型加载、内存和显存错误；
- 服务关闭与状态保存结果。

10.2 健康检查

当前桌面启动器通过 HTTP 轮询判断服务是否就绪，默认启动超时 30 秒。运维检查分三层：

层级	检查内容	通过标准
进程层	EXE、Uvicorn 线程、状态窗口	无重复进程，窗口可响应
服务层	本地 HTTP 地址	返回非 5xx 响应
功能层	导入、分离、分析、保存	核心工作流完成且结果可重载

后续可增加 `/health` 和 `/diagnostics` 端点，返回版本、运行时间、磁盘空间、模型状态和 Provider，但不得泄露用户文件内容。

10.3 服务目标

本项目是本地桌面软件，不对外提供 7x24 在线服务，因此以下指标是内部运维目标而非商业 SLA：

指标	目标
正常启动成功率	发布回归场景 100%
本地服务就绪时间	常规机器 30 秒内，不含首次大型模型推理
正常退出数据保存	100% 执行 Kernel shutdown
严重缺陷响应	发现后 1 个工作日内完成分级
阻断缺陷处置	暂停发布或回滚上一稳定版

10.4 容量与资源管理

音频分离和和弦分析可能消耗大量内存、显存和磁盘。发布说明应提示：

- CPU 推理速度依赖处理器和音频时长；
- GPU 推理需要兼容 NVIDIA 驱动；
- 仅支持 CPU 的插件即使用户选择 GPU 也会回退 CPU；
- 模型对声道布局存在差异，Bass Root Detector 需要临时单声道输入；
- 超长音频可能引发内存不足，应限制输入时长或采用分块处理；
- 缓存包含分离后的 WAV，磁盘占用可能显著高于原始音频。

11. 事件、问题和缺陷管理

11.1 事件分级

等级	定义	示例	处理目标
P0 阻断	无法启动、数据大范围损坏、发布包不可安装	EXE 全面启动失败	立即停止发布并回滚
P1 严重	核心功能不可用，无合理替代	所有分离模型失败	当日定位并给出处置
P2 一般	部分插件、特定环境或体验异常	单一和弦插件失败	纳入近期修复
P3 轻微	文案、非核心显示或优化建议	进度显示不精确	排期处理

11.2 标准处置流程

- 1 记录软件版本、安装包类型、操作系统、CPU/GPU、音频信息和时间。
- 2 收集日志和可复现步骤，隐去用户敏感信息。
- 3 判断是程序、模型、环境、数据还是资源不足问题。
- 4 确定严重度、影响范围和临时规避方式。
- 5 在独立分支修复并增加回归测试。
- 6 评审后发布补丁版本。
- 7 验证用户场景并关闭问题。
- 8 对 P0/P1 执行简化复盘，更新检查表或监控项。

11.3 常见问题处理

现象	可能原因	诊断与建议
源码启动时下载 FFmpeg 超时	<code>static_ffmpeg</code> 未找到本地 FFmpeg并访问 GitHub	配置系统 PATH 或使用发行版携带 FFmpeg
车间加载失败，提示 <code>state.json</code> 缺失	缓存残留或状态损坏	不影响其他车间；检查目录后恢复或清理
GPU 选项实际使用 CPU	插件仅支持 CPU、CUDA 不可用或 Provider 缺失	查看请求设备和实际设备日志
Bass 分析申请超大内存	双声道数组被算法误解释	仅为该插件生成临时 float32 单声道输入
GPU 升级后启动异常	只替换部分 CUDA/Torch 文件或版本不一致	使用完整 GPU 升级包覆盖运行环境
服务地址不是 8000	端口已占用	使用状态窗口显示的新地址
安装包下载不完整	GPU 分片缺失或损坏	确认 EXE/BIN 同目录并校验 SHA-256

12. 安全、合规与供应链

12.1 本地服务安全

服务默认绑定 `127.0.0.1`，不直接向局域网或互联网开放。若未来支持远程访问，必须增加身份认证、TLS、跨域限制、上传大小限制和审计日志，不能简单将 Host 改为 `0.0.0.0`。

12.2 软件供应链

第三方依赖和模型存在许可证、来源和完整性风险。正式发布前应：

- 固定关键直接依赖版本；
- 审核模型、ISMIR2019、ChordMini、FFmpeg 等许可证；
- 记录下载来源与 SHA-256；
- 使用 CodeQL 和依赖漏洞扫描；
- 禁止构建脚本在用户首次启动时静默下载可执行文件；
- 对 GitHub Actions 固定到可信版本或提交哈希。

当前 AI Review Action 使用 `@latest`，存在上游版本漂移风险，后续应固定版本。当前 EXE 尚未代码签名，Windows SmartScreen 可能提示未知发布者；正式对外发行前建议购买代码签名证书并将签名校验加入发布流程。

12.3 隐私

用户音频、分析结果和日志默认保存在本机。缺陷反馈时应由用户主动选择上传内容。团队不得将用户音频、完整本机路径或日志中的隐私信息直接附加到公开 Issue。

13. 发布、回滚与运维检查表

13.1 发布前

- ☐ 版本范围、提交哈希和负责人已确定。
- ☐ 工作区无未说明的修改，依赖和模型清单已冻结。
- ☐ Ruff、类型检查和受影响测试已执行。
- ☐ Python 与 JavaScript 核心回归已执行。
- ☐ CPU 构建环境确认无 CUDA。
- ☐ GPU 构建环境确认 CUDA 和 GPU Provider。
- ☐ FFmpeg/FFprobe 已包含且可执行。
- ☐ 基础包、GPU 包、模型包安装顺序已验证。
- ☐ 启动窗口、自动浏览器、端口回退、日志和安全退出已验证。
- ☐ 所有资产小于 2 GiB。
- ☐ `SHA256SUMS.txt` 已生成并复核。
- ☐ 许可证、系统要求、已知问题和回滚说明已完成。

13.2 发布中

- ☐ 创建受保护 Tag 和 GitHub Release 草稿。
- ☐ 上传全部 EXE、BIN 和校验清单。
- ☐ 对 Release 远端文件重新下载或抽样校验。
- ☐ 由非上传者复核文件数量、大小、哈希和说明。
- ☐ 将 Release 从草稿转为正式发布。

13.3 发布后

- ☐ 在干净 Windows 用户环境执行安装烟雾测试。
- ☐ 记录首次启动、CPU 分离、GPU 分离和模型分析结果。
- ☐ 监控 Issue、日志反馈和下载完整性问题。
- ☐ P0/P1 问题触发暂停发布或回滚。

- [] 更新开发日志、配置基线和已知问题。

14. 角色分工与 RACI

RACI 中 R 为执行, A 为最终负责, C 为协商, I 为知会。

活动	项目负责人	配置管理员	开发负责人	测试负责人	发布/运维
版本范围批准	A	C	R	C	I
配置基线建立	I	A/R	C	C	C
代码和依赖变更	I	C	A/R	C	I
测试准入	I	I	C	A/R	C
安装包构建	I	C	C	C	A/R
Release 上传	A	C	I	C	R
故障分级	A	I	C	R	R
回滚决策	A	C	C	C	R
复盘与改进	A	C	R	R	R

15. 风险登记与持续改进

风险	概率	影响	当前控制	后续措施
GPU 发行体积超过平台限制	高	高	Inno Setup 分片	自动校验和多渠道镜像
CPU/GPU 依赖混装	中	高	独立 requirements 和构建环境	CI 环境矩阵
模型缺失或版本不匹配	中	高	独立模型包和安装顺序检查	模型清单与启动诊断
用户缓存损坏	中	高	原子写入、自动保存、坏车间隔离	schema 迁移与备份工具
首次启动联网下载失败	中	中	发行版携带 FFmpeg	禁止运行期隐式下载
未签名 EXE 被拦截	高	中	发行说明提示	代码签名
CI 仅覆盖安全扫描	高	中	本地测试和发布检查表	增加 lint/test/build workflow
第三方 Action 版本漂移	中	中	PR 可见	固定 Action 提交哈希
超长音频耗尽内存	中	高	插件级输入修正	输入限制与分块推理

持续改进优先级：

- 1 建立 PR 自动 lint、测试和覆盖率流水线。
- 2 完成拆分安装包的干净机器端到端验证。
- 3 建立模型资产清单、许可证清单和 SBOM。
- 4 增加健康诊断接口、日志轮转和车间导出/恢复工具。
- 5 固定 GitHub Actions 版本并增加依赖漏洞扫描。

6 建立签名、自动 Release 草稿和远端制品复核。

16. 当前结论

TABsucks 已具备课程项目阶段较完整的配置与运维基础：Git 多分支协作、PR 自动评审、CodeQL 安全扫描、分层依赖、Windows 自动构建脚本、本地桌面启动器、日志窗口、用户数据隔离、原子状态保存、CPU/GPU 运行时验证以及面向 GitHub 2 GiB 限制的拆分发行业设计。

当前主要短板是普通 CI 流水线、代码签名、完整制品供应链记录和最终拆分安装链验证尚未全部闭环。项目组应将这些内容作为发布前准入项，而不是在文档中将其描述为已经完成。按本文档执行后，项目能够从“开发者本机可运行”演化为“配置可追踪、构建可复现、发布可审计、故障可处置”的工程化软件项目。

附录 A：关键路径

源码入口：scripts/tabsucks_launcher.py
 桌面运行时：src/ui/desktop_launcher.py
 状态窗口：src/ui/desktop_window.py
 依赖配置：requirements*.txt / constraints-windows.txt
 质量配置：pyproject.toml
 PyInstaller：packaging/tabsucks.spec
 安装包：packaging/tabsucks-*.iss
 构建脚本：scripts/build_windows.ps1
 拆分构建：scripts/build_split_release.ps1
 运行时验证：scripts/verify_packaged_runtime.py
 Windows 发行说明：docs/windows_release.md
 用户数据：%LOCALAPPDATA%\TABsucks
 安装目录：%LOCALAPPDATA%\Programs\TABsucks

附录 B：版本发布记录模板

版本号：
 发布日期：
 Git Tag：
 提交哈希：
 构建负责人：
 复核负责人：
 Python 版本：
 CPU PyTorch：
 GPU PyTorch/CUDA：
 ONNX Runtime Provider：
 FFmpeg 版本与哈希：
 模型包版本与哈希：
 测试报告：
 已知问题：
 回滚版本：
 Release 地址：

附录 C：配置审计证据

本文档主要依据以下项目证据编制：

- Git 分支、提交历史和 .gitignore;
- .github/workflows/ai-pr-reviewer.yml 与 codeql.yml;
- requirements*.txt、constraints-windows.txt 和 pyproject.toml;

- `packaging/tabsucks.spec`、Inno Setup 配置和 PowerShell 构建脚本；
- Windows 桌面启动、日志、端口探测、模型路径和安全退出代码；
- 车间缓存、原子状态保存、自动保存、回收备份及相关测试；
- 项目架构、开发指导、Windows 发行和阶段开发日志；
- CPU/GPU 选择、Bass 插件单声道兼容和拆分发行的开发与验证记录。